

Costs 'C' to divide at each step.

Date \_\_\_\_\_

Recurrence relation of Binary Search =  $\log_2(n) * C$

## LECTURE-4

Monday  
3/8/26.



Factorial:

(n-1) instead of (n/b)

$$T(n) = T(n-1) + C_1$$

checks if base case is hit in each iteration.

$$T(n) \xrightarrow{C_1} T(n-1) \xrightarrow{C_1} T(n-2) \dots T(2) \xrightarrow{C_2} T(1)$$

Terminal Nodes

★ Work done at each level will be added for total work

$$T(n) = (n-1) \cdot C_1 + C_2$$

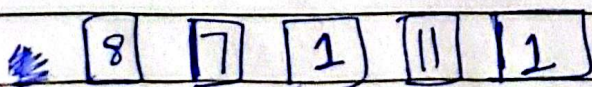
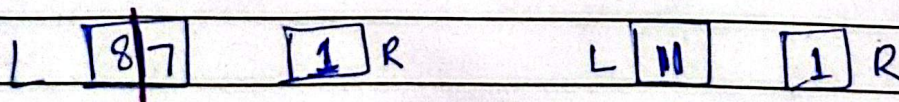
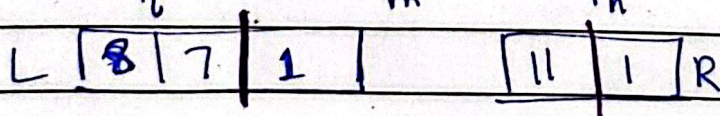
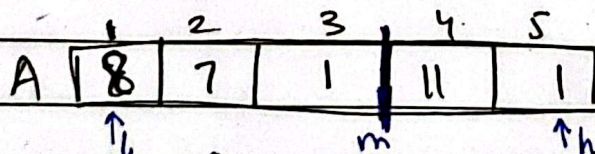
$$\therefore T(n) = \Theta(n)$$

as constants are cancelled



Merge Sort (Divide & Conquer):

Keep breaking the array from middle until we've single element in each section which makes it sorted (as single element)

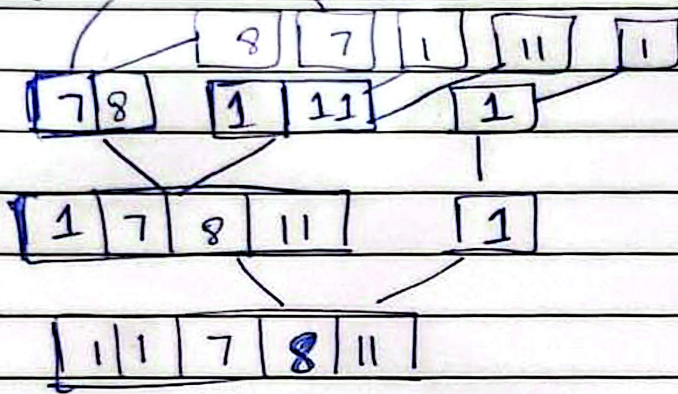


★ we'll stop when  $l=h$  in each section.



## Merging Routine

Date \_\_\_\_\_



↓ for this we'll be needing  $n_L$  and  $n_R$  (sizes of left & right) as we need to create the actual right & left arrays.  
So ;  $n_L = m - l + 1$

$$n_R = h - m$$

+ Now create L & R arrays.

Merge Sort (A [1:n], l, h) {

if (l ≥ h) return

else {

$$m = (l + h) / 2$$

Merge Sort (A, l, m)

Merge Sort (A, m+1, h)

Merge Routine (A, l, m, h) // T(n) for merge routine is  $O(n)$

as we're three single loops

}

}